

Paterni ponašanja

1. Strategy patern

Strategy patern izdvaja algoritam iz matične klase i smješta ga u posebne klase. Omogućava fleksibilan izbor algoritma bez izmjene klijentskog koda. Klijent bira odgovarajuću strategiju, a algoritmi su enkapsulirani i implementirani kroz zajednički interfejs. Ovaj patern zamjenjuje nasljeđivanje kompozicijom, što čini kod fleksibilnijim i lakšim za proširivanje.

Funkcionalnost pametnog planiranja boravka koristi algoritam koji preporučuje atrakcije, ugostiteljske objekte i evente na osnovu korisničkih preferencija, datuma boravka i interesa korisnika. Trenutno bi ovaj algoritam mogao biti smješten u jednoj klasi *SmartPlanner*, što otežava dodavanje novih načina preporučivanja i planiranja.

Primjena:

- Kreiranje interfejsa *PlanningStrategy*
- Implementacija različitih strategija: *HistoryPlanningStrategy*, *NaturePlanningStrategy*, *CulturePlanningStrategy*, *FoodPlanningStrategy*, *NightlifePlanningStrategy*
- Omogućava lako dodavanje novih algoritama planiranja bez mijenjanja postojeće klase *SmartPlanner*
- Različiti korisnici mogu koristiti različite strategije planiranja na osnovu svojih preferencija

2. State patern

State patern omogućava objektima da mijenjaju ponašanje u zavisnosti od svog unutrašnjeg stanja. Svako stanje je predstavljeno kao posebna klasa, a klasa konteksta delegira ponašanje trenutnom stanju. Ovo pojednostavljuje kod i čini ga lakšim za održavanje, posebno kada objekat može imati različita stanja koja utiču na dostupne funkcionalnosti.

Newsletter u sistemu **GoSarajevo** ima različita stanja koja utiču na dostupne akcije. Newsletter može biti u stanju nacrt, spreman za slanje, poslan ili neuspješno poslan. Trenutno bi se ova logika mogla rješavati kroz if-else provjere, što može dovesti do kompleksnog i teško održivog koda.

Primjena:

- Kreiranje stanja: *DraftNewsletterState*, *ReadyNewsletterState*, *SentNewsletterState*, *FailedNewsletterState*
- Svako stanje definiše koje funkcionalnosti su dostupne za newsletter
- Automatski prelazak između stanja kada se newsletter pripremi, pošalje ili slanje ne uspije
- Pojednostavljuje upravljanje newsletterom na osnovu njegovog statusa

3. Template Method patern

Template Method patern definiše kostur algoritma u nadklasi, a prepušta podklasama da implementiraju specifične korake. Pruža mogućnost ponovne upotrebe koda i prilagodbu dijelova algoritma. Ovaj patern je povezan sa nasljeđivanjem, te nadklasa kontroliše tok izvršavanja i poziva metode podklasa kada je potrebno.

Kreiranje različitih turističkih sadržaja u sistemu **GoSarajevo** dijeli sličnu strukturu. Kreiranje eventa, atrakcije i ugostiteljskog objekta zahtijeva unos osnovnih podataka, validaciju, postavljanje koordinata, dodavanje slike i spremanje u bazu. Trenutno se ova logika može duplicirati za svaki tip sadržaja posebno.

Primjena:

- Kreiranje nadklase *ContentCreationTemplate* sa metodom *CreateContent()*
- Podklase *EventCreation*, *AttractionCreation* i *HospitalityCreation* implementiraju specifične korake
- Omogućava ponovnu upotrebu zajedničke logike za kreiranje sadržaja
- Lakše dodavanje novih tipova turističkog sadržaja u budućnosti

4. Observer patern

Observer patern uspostavlja jednosmjernu zavisnost između objekata tako da promjena stanja u jednom objektu automatski obavještava sve njegove posmatrača. Koristan je za implementaciju događajnog sistema gdje jedan objekat može imati više posmatrača koji se automatski ažuriraju kada se stanje promijeni. Ovaj patern omogućava skalabilnost i odvojenost subjekta i posmatrača.

Kada se u sistemu **GoSarajevo** doda novi event, atrakcija ili ugostiteljski objekat, potrebno je obavijestiti različite dijelove sistema - registrovane korisnike, newsletter servis i SmartPlanner preporuke. Trenutno se ove akcije mogu pozivati direktno iz koda koji mijenja sadržaj, što povećava zavisnost između klasa.

Primjena:

- Kreiranje *TourismContentSubject* koji obavještava posmatrača o promjenama sadržaja
- Implementacija observera: *NewsletterObserver*, *EmailNotificationObserver*, *SmartPlannerUpdateObserver*
- Omogućava automatsko obavještavanje korisnika o novim eventima, atrakcijama i ugostiteljskim objektima
- Omogućava dodavanje novih funkcionalnosti koje reaguju na promjene bez mijenjanja postojećeg koda

5. Iterator patern

Iterator patern omogućava sekvencijalni pristup elementima kolekcije bez otkrivanja njene interne strukture. Podržava više istovremenih iteratora nad istom kolekcijom. Klijent koristi zajednički interfejs, bez potrebe da zna konkretni tip kolekcije. Ovaj patern čini kod nezavisnim od konkretne implementacije kolekcije i omogućava jednostavno iteriranje kroz različite tipove struktura podataka.

Pregled različitih kolekcija u sistemu **GoSarajevo**, kao što su event, atrakcija, ugostiteljski objekti, recenzije i preporuke u SmartPlanneru, zahtijeva sekvencijalni pristup elementima. Trenutno bi se za prolazak kroz ove kolekcije mogle koristiti obične petlje, što čini kod zavisnim od konkretnog tipa kolekcije.

Primjena:

- Kreiranje iteratora za različite kolekcije: *EventIterator*, *AttractionIterator*, *HospitalityIterator*, *ReviewIterator*
- Omogućava jedinstveni pristup iteriranju bez obzira na tip kolekcije
- Podržava više istovremenih iteratora nad istom kolekcijom
- Olakšava dodavanje filtriranja i sortiranja sadržaja
- Koristi se kod pregleda sadržaja i generisanja preporuka u SmartPlanneru

6. Command patern

Command patern enkapsulira zahtjev kao objekat, čime omogućava parametarsko pokretanje operacija, kao i mogućnost undo/redo, reda čekanja i bilježenja akcija. Ovaj patern razdvaja pošiljaoca od izvršioca akcije i koristi se gdje je potrebno odvojeno upravljanje operacijama. Komande mogu biti pohranjene, proslijeđene kao parametri i izvršene u različito vrijeme.

Admin funkcionalnosti u sistemu **GoSarajevo**, kao što su upravljanje atrakcijama, eventima, ugostiteljskim objektima i recenzijama, zahtijevaju jasno odvojene operacije dodavanja, brisanja i ažuriranja. Također, neke operacije poput brisanja sadržaja ili izmjene podataka trebaju biti reverzibilne ili zabilježene radi praćenja aktivnosti administratora.

Primjena:

- Enkapsulacija svake admin operacije kao komande: *AddAttractionCommand*, *UpdateEventCommand*, *DeleteHospitalityCommand*, *DeleteReviewCommand*
- Implementacija *CommandInvoker* klase za izvršavanje komandi
- Mogućnost čuvanja historije komandi za undo/redo funkcionalnost
- Lakše praćenje administratorskih akcija i promjena nad podacima u sistemu

7. Chain of Responsibility patern

Chain of Responsibility patern predstavlja lanac objekata kroz koji prolazi zahtjev. Ako jedan objekat ne može obraditi zahtjev, prosljeđuje ga narednom objektu u lancu. Ovaj patern smanjuje zavisnost između pošiljaoca zahtjeva i konkretnog objekta koji zahtjev obrađuje.

Ovaj patern se može koristiti kod obrade korisničkih zahtjeva ili validacije podataka. Na primjer, prilikom dodavanja novog eventa, atrakcije ili ugostiteljskog objekta, zahtjev može proći kroz više koraka: provjeru obaveznih polja, provjeru koordinata, provjeru slike i provjeru prava pristupa administratora.

Primjena:

- Kreiranje lanca validacije: *RequiredFieldsHandler*, *CoordinatesValidationHandler*, *ImageValidationHandler*, *AdminPermissionHandler*
- Svaki handler provjerava jedan dio zahtjeva

- Ako je zahtjev ispravan, proslijeđuje se narednom handleru
- Olakšava dodavanje novih provjera bez mijenjanja postojeće logike

8. Mediator patern

Mediator patern enkapsulira komunikaciju između više objekata u posebnu posredničku klasu. Umjesto da objekti međusobno direktno komuniciraju, oni komuniciraju preko medijatora. Ovaj patern smanjuje zavisnost između klasa i čini sistem preglednijim.

Mediator patern se može koristiti kod komunikacije između SmartPlannera, korisničkih preferencija, evenata, atrakcija i ugostiteljskih objekata. Umjesto da SmartPlanner direktno komunicira sa svim klasama, komunikacija se može organizovati preko posebne klase koja koordinira proces generisanja plana.

Primjena:

- Kreiranje klase *SmartPlannerMediator*
- Mediator povezuje Preferencija, Event, Atrakcija i UgostiteljskiObjekat
- SmartPlanner preko medijatora dobija potrebne podatke za generisanje plana
- Smanjuje direktnu zavisnost između klasa
- Olakšava održavanje logike pametnog planiranja

9. Visitor patern

Visitor patern omogućava dodavanje novih operacija nad postojećim objektima bez mijenjanja njihovih klasa. Nova operacija se izdvaja u posebnu visitor klasu, a objekti prihvataju visitor koji izvršava određenu funkcionalnost.

Visitor patern se može koristiti nad različitim tipovima turističkog sadržaja, kao što su Event, Atrakcija i UgostiteljskiObjekat. Na primjer, sistem može imati operaciju za izračunavanje popularnosti, generisanje izvještaja ili pripremu podataka za prikaz korisniku, bez mijenjanja samih klasa sadržaja.

Primjena:

- Kreiranje interfejsa *TourismContentVisitor*

- Implementacija visitor klasa: *PopularityVisitor*, *ReportVisitor*, *MapDisplayVisitor*
- Klase Event, Atrakcija i UgostiteljskiObjekat prihvataju visitora
- Omogućava dodavanje novih operacija bez izmjene postojećih modela
- Korisno za izvještaje, statistiku i prikaz sadržaja

10. Interpreter patern

Interpreter patern se koristi za interpretaciju jednostavnog jezika ili pravila definisanih u sistemu. Omogućava da se određeni izrazi, filteri ili korisnički uslovi predstave kroz posebne klase koje ih tumače.

Ovaj patern se može koristiti kod filtriranja preporuka na osnovu korisničkih preferencija. Korisnik može imati pravila kao što su: prikaži historijske atrakcije, prikaži objekte tradicionalne hrane ili prikaži evente koji se održavaju tokom boravka. Ova pravila se mogu interpretirati i koristiti za filtriranje sadržaja.

Primjena:

- Kreiranje interfejsa *Expression*
- Implementacija izraza: *HistoryExpression*, *FoodExpression*, *DateRangeExpression*, *CultureExpression*
- Svaki izraz interpretira jedno pravilo filtriranja
- Omogućava fleksibilnije filtriranje sadržaja
- Koristi se kod SmartPlanner preporuka i pregleda sadržaja

11. Memento patern

Memento patern omogućava čuvanje prethodnog stanja objekta, kako bi se to stanje kasnije moglo vratiti. Koristan je kada sistem treba podržati poništavanje izmjena ili vraćanje na prethodnu verziju podataka.

U sistemu **GoSarajevo** Memento patern se može koristiti kod uređivanja SmartPlanner plana ili administratorskog uređivanja sadržaja. Na primjer, korisnik može izmijeniti svoj plan boravka, ali se kasnije vratiti na prethodnu verziju plana. Također, administrator može izmijeniti podatke o eventu ili atrakciji, a sistem može sačuvati prethodno stanje radi mogućeg vraćanja.

Primjena:

- Kreiranje klase *SmartPlannerMemento*
- Kreiranje klase *SmartPlannerHistory*
- Čuvanje prethodnih verzija plana boravka
- Omogućava vraćanje plana na prethodno stanje
- Korisno za undo funkcionalnost kod izmjene plana ili sadržaja